

Here's a polished **LeetCode-style** version of the problem that closely matches what you've described from the Harness OA.

Maximum Weight Tree After Degree Constraints

Difficulty: Hard

You are given an undirected weighted tree consisting of n servers numbered from 0 to $n - 1$.

The tree is represented using three integer arrays:

- $g_from[i]$ and $g_to[i]$ represent an undirected edge between two servers.
- $g_weight[i]$ represents the weight of that edge.

You are also given an integer k .

You may remove any number of edges from the tree. After removing the edges, every server must have **at most k incident edges**.

Return the **maximum possible sum of the weights of the remaining edges**.

Example 1

Input

```
n = 5

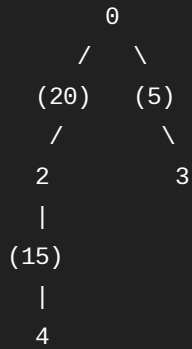
g_from = [0,0,0,2]
g_to   = [1,2,3,4]
g_weight = [10,20,5,15]

k = 2
```

Explanation

Initially,

```
  1
  |
(10)
  |
```



Server `0` has degree `3`, which exceeds `k`.

Remove edge `(0,3)` having weight `5`.

The remaining edges have total weight

$$10 + 20 + 15 = 45$$

Output

45

Example 2

Input

```
n = 4

g_from = [0,1,2]
g_to   = [1,2,3]
g_weight = [4,7,5]

k = 2
```

Output

16

Explanation

Every node already satisfies the degree constraint.

No edge needs to be removed.

Example 3

Input

```
n = 6

g_from   = [0,0,0,1,2]
g_to     = [1,2,3,4,5]
g_weight = [8,6,10,5,9]

k = 1
```

Output

```
19
```

Explanation

One optimal solution is to keep edges

```
0 - 3 (10)
2 - 5 (9)
```

All nodes have degree at most **1**, and the total remaining weight is **19**.

Function Signature

C++

```
class Solution {
public:
    long long maximumWeight(
        int n,
        vector<int>& g_from,
        vector<int>& g_to,
        vector<int>& g_weight,
        int k
    );
};
```

Constraints

```
2 <= n <= 2 × 105
```

```
g_from.length == g_to.length == g_weight.length == n - 1
```

```
0 <= g_from[i], g_to[i] < n
```

```
1 <= g_weight[i] <= 109
```

```
1 <= k <= n - 1
```

The given edges form a valid tree.

Notes

- The graph is **undirected**.
- You may remove **zero or more** edges.
- After all removals, every node must have **degree $\leq k$** .
- Your objective is to **maximize the total weight of the remaining edges**.

This is an **$O(N \log N)$** tree DP problem. The expected approach is to root the tree, perform a post-order DFS, and use dynamic programming with two states (`dp0` and `dp1`) to decide which child edges to keep while respecting the degree constraint. This is the same approach commonly used for degree-constrained tree optimization problems.